The artifact Wrapify was created earlier this year as a personal project to practice my python coding and to build something that was interesting and meaningful to me. The original version of the code prior to enhancements would take the folder of JSONs that Spotify provides to users who request their data and concatenate each file to build one large csv.

I selected this item because it shows my ability to create a self-directed project and find unique ways to solve problems. One such problem I solved includes Spotify's method of data collection, which includes making multiple JSON files with many variables, which can be difficult to query. I solved this problem by building a large JSON out of the many smaller JSONs and turning it into a CSV to make future functions easier to build. The artifact was improved mostly by simplifying some overly complicated code, but also by making results far more legible to users. I also went through and finished a lot of the basic functionality planned for this project (top songs/artists/albums by year, etc.) so in its current state, it is perfectly usable.

The course out comes I met include "Employ strategies for building collaborative environments that enable diverse audiences to support organizational decision making in the field of computer science." I met this outcome by ensuring Wrapify works for anyone with their Spotify data, not just my dataset alone. Should you request your Spotify data and load it into the same directory as Wrapify, you will be able to read your Spotify listening history results. I also met "Demonstrate an ability to use well-founded and innovative techniques, skills, and tools in computing practices for the purpose of implementing computer solutions that deliver value and accomplish industry-specific goals." I met this outcome by addressing a common desire among end users, which is to read their metrics and data each year. I took an idea that already existed, Spotify Wrapped, and changed it to include some more specific values such as an increased list

of favorite songs and artists, and specific counts for each one. I also made use of libraries available to python programming that make data visualization much easier.

I had planned to meet: “Develop a security mindset that anticipates adversarial exploits in software architecture and designs to expose potential vulnerabilities, mitigate design flaws, and ensure privacy and enhanced security of data and resources.” But I think I fell short on this one. I have some error-handling in the beginning but not so much throughout the course of the program.

When enhancing my artifact, the biggest issue I faced was with visualization. The output, in its original format, was hardly legible to me, much less someone who didn’t build the project themselves. I did some research on what libraries are best for visualizing datasets, and my data was especially tricky since very few variables are numeric, but my research led me to Seaborn which is great for visualizing statistical data. I could use their anagram function to put my data in easy-to-read cells, and their bar plot function to make correlation between counts easier to read for visual learners. Much of the rest of my project went very smoothly, since I built my [R]apify project prior to this one, I had a lot of methods already figured out and just reapplied them for this project.

I did, however, learn why the R language is so heavily utilized for datasets. As I was working and referencing my own work, I realized how few lines I needed to write in R to get my desired output, but for Python I felt like I was writing miles of code to do the same thing. That was an interesting observation to experience firsthand. I also realized, more towards the end of my planned functionality, that I could’ve made everything far simpler if I made use of more methods and called those, rather than hard coding everything. As of right now, my project does exactly as I originally intended, but perhaps in the future, I will go through and make the “front

facing" code more succinct and have a separate file of definitions and methods that has the more redundant script.